



PROGRAMACION II - POO

BARAJA DE CARTAS ESPANOLAS - SEGUNDA PARTE DEL PROYECTO

Implementacion de un Juego con Naipes Espanoles: Solitario Espanol (Modo Clasico y Modo Dificil)

Materia: Programacion II - Programacion Orientada a Objetos

Profesor: David Roco 

Capitan del equipo: [Mariano Chirino] ★

Integrantes: 

[Andres Fabre]

[Facundo Quiroga]

Fecha de entrega: 22 de junio de 2026



1. Introduccion

En la primera etapa del proyecto se diseño e implemento una Baraja de Cartas Espanolas utilizando Programacion Orientada a Objetos y ArrayList, con las clases Carta y Baraja como base del sistema. En esta segunda etapa, el equipo utilizo esa baraja para implementar un juego completo: el Solitario Espanol, respetando las reglas oficiales del juego y agregando un modo de dificultad adicional disenado por el equipo. El objetivo de este informe es documentar el analisis, el disenyo y la implementacion realizados, asi como los resultados de las pruebas de balance jugadas durante el desarrollo.

2. Juego Elegido

El equipo eligio implementar el Solitario Espanol, un juego de un solo jugador en el que se deben formar parejas de cartas del mismo numero para vaciar completamente la mano, la mesa y la baraja. Se eligio este juego porque permite aplicar de forma clara los conceptos de Programacion Orientada a Objetos trabajados en la materia (clases, colecciones con ArrayList, separacion de responsabilidades) sin requerir logica de multiples jugadores simultaneos, lo que facilito enfocar el trabajo en el disenyo de las reglas, la reutilizacion de las clases de la primera etapa y la incorporacion de un sistema de dificultad propio.



3. Reglas del Juego

3.1 Modo Clasico (reglas oficiales)

Objetivo: el jugador debe eliminar todas las cartas de la baraja, de su mano y de la mesa, formando parejas del mismo numero. El palo de las cartas no importa para formar parejas.

Preparacion:

- Se usan 40 cartas de la baraja espaniola (se excluyen el 8 y el 9 de cada palo).
- Se mezclan las cartas aleatoriamente.
- Se colocan 5 cartas boca arriba en la mesa.
- El jugador recibe 3 cartas en su mano.
- Las 32 cartas restantes forman la baraja de robo.

Desarrollo del turno.

En cada turno el jugador elige una carta de su mano y la juega. Pueden darse tres situaciones:

- Hay una carta del mismo numero en la mesa: ambas cartas se eliminan del juego y el jugador roba 1 carta.
- Hay otra carta del mismo numero en la mano: ambas cartas de la mano se eliminan del juego y el jugador roba 2 cartas.
- No hay pareja ni en la mesa ni en la mano: la carta jugada se agrega al final de la fila de la mesa y el jugador roba 1 carta.

El jugador siempre mantiene 3 cartas en mano al inicio de cada turno, mientras haya cartas disponibles en la baraja. Si la baraja esta vacia, no se roba.

Victoria: el jugador gana cuando la baraja, la mano y la mesa estan completamente vacias al mismo tiempo.

Derrota: el jugador pierde cuando la baraja se agota y no existen parejas posibles entre la mano y la mesa.

Aclaraciones:

- Si hay mas de una carta del mismo numero en la mesa, se elimina solo UNA con la carta jugada.
- La derrota solo se declara cuando no hay ninguna jugada posible.
- El jugador no puede pasar su turno sin jugar una carta.
- No hay limite de cartas que pueden estar en la mesa.



3.2 Modo Difícil (mecánica adicional del equipo)



El modo difícil usa la misma base que el modo clásico, pero con las siguientes diferencias, diseñadas y balanceadas por el equipo durante varias rondas de testeo:

- El jugador recibe 3 cartas en mano y 5 cartas en mesa al inicio (igual que el modo clásico).
- El jugador cuenta con 6 vidas.
- La mesa tiene un límite máximo de 10 cartas.

Regla de pareja restrictiva:



Una pareja no se forma solo por tener el mismo número. Además del número, las cartas deben pertenecer al mismo grupo de palo:

- Grupo A: Oros y Copas
- Grupo B: Espadas y Bastos

Por ejemplo, un 5 de Oros solo puede emparejar con un 5 de Copas. Un 5 de Espadas solo puede emparejar con un 5 de Bastos. Un 5 de Oros NO empareja con un 5 de Espadas, porque pertenecen a grupos distintos.

Mesa llena:

Si la mesa llega a su límite de 10 cartas y el jugador juega una carta que no forma pareja ni con la mesa ni con la mano, ocurre lo siguiente:

- La carta jugada vuelve al fondo de la baraja (no entra a la mesa).
- El jugador pierde una vida.
- Toda la mesa se renueva: las cartas actuales de la mesa se envían al fondo de la baraja y se colocan cartas nuevas tomadas del tope de la baraja.

Si el jugador se queda sin vidas, pierde la partida. La condición de victoria es la misma que en el modo clásico.

4. Analisis del Problema



1. Que juego eligieron y por que?

Se eligio el Solitario Espanol porque es un juego de un solo jugador que permite concentrar el trabajo en el diseno de clases y en las reglas, sin la complejidad adicional de gestionar turnos entre varios jugadores.

2. Cuales son las reglas principales del juego?

Formar parejas de cartas del mismo numero (en modo clasico) o del mismo numero y mismo grupo de palo (en modo dificil) para vaciar la mano, la mesa y la baraja. El detalle completo esta en la seccion 3.

3. Que clases nuevas necesitaron agregar?

Jugador, Mesa, MenuConsola, Partida, ReglaJuego y Resultado, ademas de la clase Main como punto de entrada del programa.

4. Que clases de la primera parte reutilizaron?

Carta y Baraja se reutilizaron tal como fueron disenadas en la primera etapa del proyecto, sin modificar su estructura interna ni sus responsabilidades originales.

5. Que responsabilidades tiene cada clase?

Ver el detalle completo en la tabla de la seccion 5.

6. Que objetos participan durante una partida?

Una instancia de Baraja, una instancia de Jugador, una instancia de Mesa, una instancia de ReglaJuego (configurada en modo clasico o dificil), una instancia de Resultado y una instancia de MenuConsola, todas coordinadas por una instancia de Partida.

7. Como se determina el ganador?

La clase ReglaJuego evalua, despues de cada jugada, si la baraja, la mano y la mesa estan vacias al mismo tiempo (victoria), o si la baraja esta vacia y no existen jugadas posibles entre la mano y la mesa, o si el jugador se quedo sin vidas en modo dificil (derrota). El resultado se registra en la clase Resultado.

8. Que patron de diseno podria ayudar a organizar mejor la solucion?

Se analizo el patron Strategy para representar la regla de formacion de parejas. El detalle de este analisis se desarrolla en la seccion 7.

5. Clases del Sistema y Responsabilidades

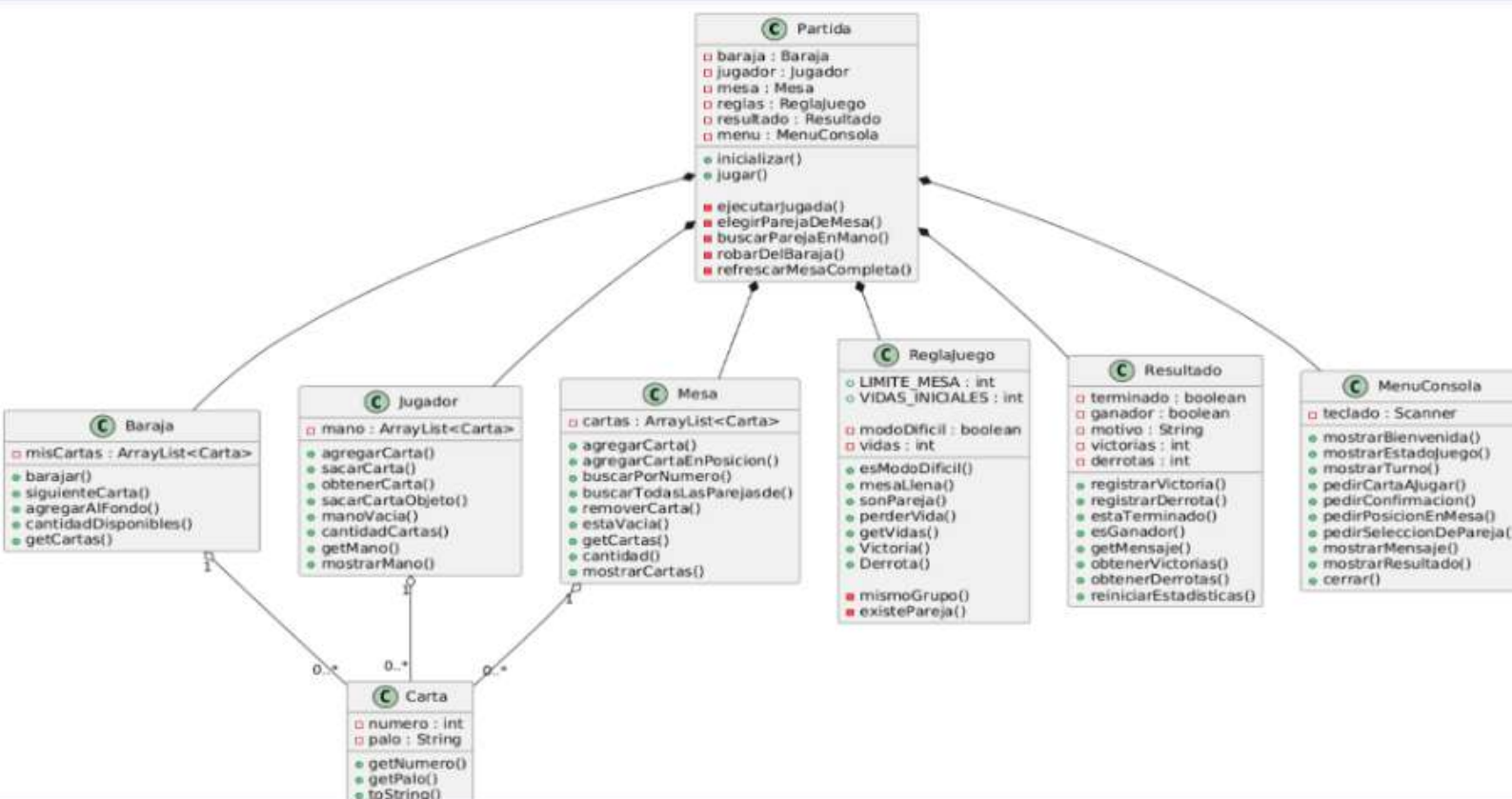
Clase	Responsabilidad principal	Origen
Carta	Representa una carta individual (numero y palo).	Reutilizada
Baraja	Gestiona el mazo completo: carga, mezcla, robo y <u>devolucion</u> de cartas al fondo.	Reutilizada
Jugador	Gestiona la mano del jugador: agregar, sacar y consultar cartas.	Nueva
Mesa	Gestiona las cartas visibles en la mesa.	Nueva
<u>ReglaJuego</u>	Define las reglas de pareja, victoria, derrota y la <u>configuracion</u> de cada modo (<u>clasico/dificil</u>).	Nueva
Partida	Coordina el flujo de un juego completo: inicializa, gestiona turnos y aplica las consecuencias de cada jugada.	Nueva
Resultado	Registra si la partida termino, si fue victoria o derrota, el motivo, y las <u>estadisticas</u> acumuladas.	Nueva
<u>MenuConsola</u>	Gestiona toda la <u>interaccion</u> por consola: muestra el estado del juego y solicita datos al jugador.	Nueva
<u>Main</u>	Punto de entrada del programa. Gestiona el <u>menu</u> principal y el ciclo de partidas.	Nueva



6. Diagrama UML



A continuacion se detalla la estructura de clases utilizada como base para el diagrama UML del proyecto (atributos, metodos principales y relaciones).





7. Patrones de Diseño Analizados

Strategy (analizado):



La logica que determina si dos cartas forman pareja cambia segun el modo de juego (clasico: mismo numero; dificil: mismo numero y mismo grupo de palo). Esto es, conceptualmente, un caso de uso tipico del patron Strategy: una familia de algoritmos intercambiables (la regla de pareja) que varia segun el contexto (el modo elegido).

En la implementacion actual, esta variacion se resolvio con un atributo booleano `modoDificil` dentro de `ReglaJuego` y una condicion `if/else` en el metodo `sonPareja()`. El equipo evaluo refactorizar esto hacia una interfaz `EstrategiaPareja` con dos implementaciones (`ReglaParejaClasica` y `ReglaParejaDificil`), inyectadas en `ReglaJuego` segun el modo elegido. Se decidio mantener la version actual con el booleano por una cuestion de alcance y tiempo de entrega, dejando documentada la alternativa con Strategy como una mejora identificada para una proxima iteracion del proyecto.

Template Method (considerado):



El metodo `ejecutarJugada()` de la clase `Partida` sigue siempre la misma secuencia de pasos (elegir carta, confirmar, buscar pareja en mesa, buscar pareja en mano, resolver mesa llena, robar cartas). Se identifico que esta secuencia fija de pasos es similar a la idea de Template Method, aunque no se llego a extraer una clase abstracta con pasos redefinibles, ya que el proyecto solo tiene dos variantes de juego (clasico y dificil) y la diferencia entre ambas se resuelve enteramente a traves de `ReglaJuego`, sin necesidad de alterar la secuencia de pasos en si.





8. Capturas de Ejecucion

Ejemplo 1 - Inicio de partida en Modo Dificil:

```

Run (Main) ...X
--- exec:3.1.0:exec (default-ell) @ Barajaespanola ---

SOLITARIO ESPANOL POR DESCARTE.

=====
----- MENU PRINCIPAL -----
1) Jugar Modo Clasico
2) Jugar Modo Dificil
3) Salir
Elegi una opcion: 2
=====

SOLITARIO ESPANOL
=====

*** MODO DIFICIL ACTIVADO: mesa limite 10 cartas, vidas: 6, mano reducida a 3 cartas ***
*** Nota: Si te pasas de las 10 cartas, la ultima carta que intentaste poner vuelve al mazo, perdes una vida y toda la mesa se renueva con ca

=====
Mesa:
[0] 11 de Bastos [1] 6 de Bastos [2] 11 de Oros [3] 10 de Copas [4] 5 de Copas

Mano:
[0] 1 de Bastos
[1] 12 de Copas
[2] 2 de Bastos

Cartas restantes en baraja: 32
Vidas: 6
Reglas Modo Dificil: pareja = mismo numero Y mismo grupo de palo
Grupo A: Oros <-> Copas | Grupo B: Espadas <-> Bastos

Ingrese el indice de la carta a jugar:
  
```

Ejemplo 2 - Pareja encontrada y resultado final (Victoria):

```

No hay mas cartas en la baraja.

=====
Mesa:
[0] 3 de Espadas

Mano:
[0] 3 de Oros

Cartas restantes en baraja: 0

Ingrese el indice de la carta a jugar: 0

Carta elegida: 3 de Oros
Confirma esta jugada? (S/N): s
[OK] PAREJA ENCONTRADA! Se eliminan: 3 de Espadas y 3 de Oros
No hay mas cartas en la baraja.

=====
GANASTE!!!
Ganaste la partida!

Estadisticas:
Victorias: 1
Derrotas: 0

=====
----- MENU PRINCIPAL -----
1) Jugar Modo Clasico
2) Jugar Modo Dificil
3) Salir
  
```




9. Conclusion del Equipo

El desarrollo de esta segunda etapa permitio comprobar en la practica las ventajas de un buen diseno orientado a objetos: las clases Carta y Baraja, construidas en la primera etapa del proyecto, se reutilizaron sin cambios para construir un juego completo, simplemente agregando nuevas clases responsables de la logica especifica del Solitario Espanol.

Un aprendizaje importante del equipo fue que la implementacion del modo dificil, pensado inicialmente como una mejora opcional, requirio varias rondas de testeo y ajuste de parametros (limite de mesa, cantidad de vidas, cantidad de cartas en mano y la propia regla de formacion de pareja) hasta lograr una dificultad pareja: desafiante, pero ganable. Esto mostro en la practica como pequenos cambios en las reglas de un juego pueden tener un impacto grande en la experiencia final, y la importancia de separar la logica de reglas (ReglaJuego) del resto del sistema para poder iterar sobre ella sin afectar el resto del codigo.

